AMRUTA INSTITUTE OF ENGINEERING AND MANAGEMENT SCIENCES – POLYTECHNIC

Course: Database System Concepts and PL/SQL

Course Code: 20CS34P

Week 3

Syllabus: Relational model: Overview; characteristics; Constraints: types; Operations; Advantages and Disadvantages; applications; Design anomalies; Features of good DB design;

Relational Model

- Relational Data Model was first prosed by **Ted Codd** of IBM in the **1970s**. Its commercial implementations were observed in the **1980s**.
- The relational data model is employed for **storing and processing** the data in the database.
- **Relational Model** represents the database as a collection of relations.
- A relation is nothing but a table of values.

Basic Terminologies Used in Relational Data Model

- **Attribute:** Each column in a Table. Attributes are the properties which define a relation. e.g., Student_Rollno, NAME,etc.
- **Tables** – In the Relational model the, relations are saved in the table format. It is stored along with its entities. A table has two properties rows and columns. Rows represent records and columns represent attributes.
- **Tuple** – It is nothing but a single row of a table, which contains a single record.
- **Relation Schema:** A relation schema represents the name of the relation with its attributes.
- **Degree:** The total number of attributes which in the relation is called the **degree** of the relation.
- **Cardinality:** Total number of rows present in the Table.
- **Column:** The column represents the set of values for a specific attribute.

- **Relation instance** – Relation instance is a finite set of tuples in the RDBMS system. Relation instances never have duplicate tuples.
- **Relation key** – Every row has one, two or multiple attributes, which is called relation key.
- **Attribute domain** – Every attribute has some pre-defined value and scope which is known as attribute domain.
- Figure shows an example of a STUDENT relation, which corresponds to the STUDENT schema just specified. Each tuple in the relation represents a particular student entity.
    - **The attributes and tuples of a relation STUDENT.**

Relation Name

Attributes

STUDENT

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	(817)373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	(817)375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	(817)749-1253	25	3.53
Rohan Panchal	489-22-1100	(817)376-9821	265 Lark Lane	(817)749-6492	28	3.93
Barbara Benson	533-69-1238	(817)839-8461	7384 Fontana Lane	NULL	19	3.25

Tuples

The attributes and tuples of a relation STUDENT.

Characteristics:

- Each **relation** in a database must have a **distinct or unique name** which would separate it from the other relations in a database.
- A relation must **not** have **two attributes** with the **same name**. Each attribute must have a distinct name.
- Duplicate tuples must **not** be present in a relation.

Roll_No.	Name	Department
101	Steive	Comp. Sci.
265	Jhoson	Finance
505	Margret	Biology
505	Margret	Biology
256	Davis	Comp. Sci.
453	Sheryl	Biology
365	Emma	Maths

Student Relation in Relational Model

- Each tuple must have exactly **one data value** for an **attribute**. For example, below in the first table, you can see that for Roll_No. 265 we have enrolled two students Jhoson and Charles, this would not work. We must have only one student for one Roll_No.

Roll_No.	Name	Department
101	Steive	Comp. Sci.
265	Jhoson Charles	Finance
505	Margret	Biology
325	Jenny	Social Sci.
256	Davis	Comp. Sci.
453	Sheryl	Biology
365	Emma	Maths

Roll_No.	Name	Department
101	Steive	Comp. Sci.
265	Jhoson	Finance
346	Charles	Finance
505	Margret	Biology
325	Jenny	Social Sci.
256	Davis	Comp. Sci.
453	Sheryl	Biology
365	Emma	Maths

- Tuples in a relation do not have to follow a significant order as the relation is not order-sensitive.

- The attributes of a relation also do not have to follow certain ordering, it's up to the developer to decide the ordering of attributes.

Constraints: types

The state of the whole database will correspond to the states of all its relations at a particular point in time. There are generally many restrictions or **constraints** on the actual values in a database state.

Constraints on databases can generally be divided into three main categories:

1. Constraints those are inherent in the data model. We call these **inherent model-based constraints** or **implicit constraints**. For example, a relation in a database must not have duplicate tuples, there is no constraint in the ordering of the tuples and attributes.
2. Constraints that can be directly expressed in schemas of the data model, typically by specifying them in the DDL. We call these **schema-based constraints** or **explicit constraints** and constraints on Null Value. They are further categorized as **domain constraints, key constraints, entity integrity constraints, referential integrity constraints.**
3. Constraints that cannot be directly expressed in the schemas of the data model, and hence must be expressed and enforced by the application programs. We call these **application-based** or **semantic constraints** or **business rules**.

The **schema-based constraints** include

- Domain constraints,
- Key constraints and constraints on NULLs,
- Entity integrity constraints, and
- Referential integrity constraints.

Domain Constraints

- Domain constraints specify that within each tuple, the value of each attribute A must be an atomic value from the domain dom(A).
- The data types associated with domains typically include standard numeric data types for integers (such as short integer, integer, and long integer) and real numbers (float and double precision float). Characters, Booleans, fixed-length strings, and variable-length strings are also available, as are date, time, timestamp, and money, or other special data types.
- Other possible domains may be described by a subrange of values from a data type or as an enumerated data type in which all possible values are explicitly listed.

Key Constraints and Constraints on NULL Values

- A relation is defined as a set of tuples and all elements of a set are distinct; hence, all tuples in a relation must also be distinct.
- This means that no two tuples can have the same combination of values for all their attributes.
- A relation schema R with the property that no two tuples in any relation state r of R should have the same combination of values for these attributes.
- Suppose that we denote one such subset of attributes by SK; then for any two distinct tuples t1 and t2 in a relation state r of R, we have the constraint that:

$$t1[SK] \neq t2[SK]$$

- Any such set of attributes SK is called a **super key** of the relation schema R. A super key SK specifies a uniqueness constraint that no two distinct tuples in any state r of R can have the same value for SK. Every relation has at least one default superkey—the set of all its attributes.
- A super key can have redundant attributes, however, a key, which has no redundancy.

- A **key** K of a relation schema R is a super key of R with the additional property that removing any attribute A from K leaves a set of attributes K_ that is not a superkey of R anymore.

Hence, a key satisfies two properties:

1. Two distinct tuples in any state of the relation cannot have identical values for (all) the attributes in the key. This first property also applies to a super key.
2. It is a minimal super key—that is, a superkey from which we cannot remove any attributes and still have the uniqueness constraint in condition 1 hold.

The attribute set {Ssn} is a key of STUDENT because no two student tuples can have the same value for Ssn. Any set of attributes that includes Ssn—for example, {Ssn, Name, Age}—is a superkey. However, the super key {Ssn, Name, Age} is not a key of STUDENT because removing Name or Age or both from the set still leaves us with a super key.

A relation schema may have more than one key. In this case, each of the keys is called a **candidate key**. One of the candidate keys is designated to be the **primary key** of the relation and others are called secondary keys.

Entity integrity constraint

The **entity integrity constraint** states that no primary key value can be NULL. This is because the primary key value is used to identify individual tuples in a relation. Having NULL values for the primary key implies that we cannot identify some tuples.

For example, if two or more tuples had NULL for their primary keys, we may not be able to distinguish them.

Referential Integrity Constraints, and Foreign Keys

Key constraints and entity integrity constraints are specified on individual relations.

The **referential integrity constraint** is specified between two relations and is used to maintain the consistency among tuples in the two relations. Informally, the referential integrity constraint states that a tuple in one relation that refers to another relation must refer to an existing tuple in that relation.

For Example: The attribute Dno of EMPLOYEE gives the department number for which each employee works; hence, its value in every EMPLOYEE tuple must match the Dnumber value of some tuple in the DEPARTMENT relation.

Foreign key

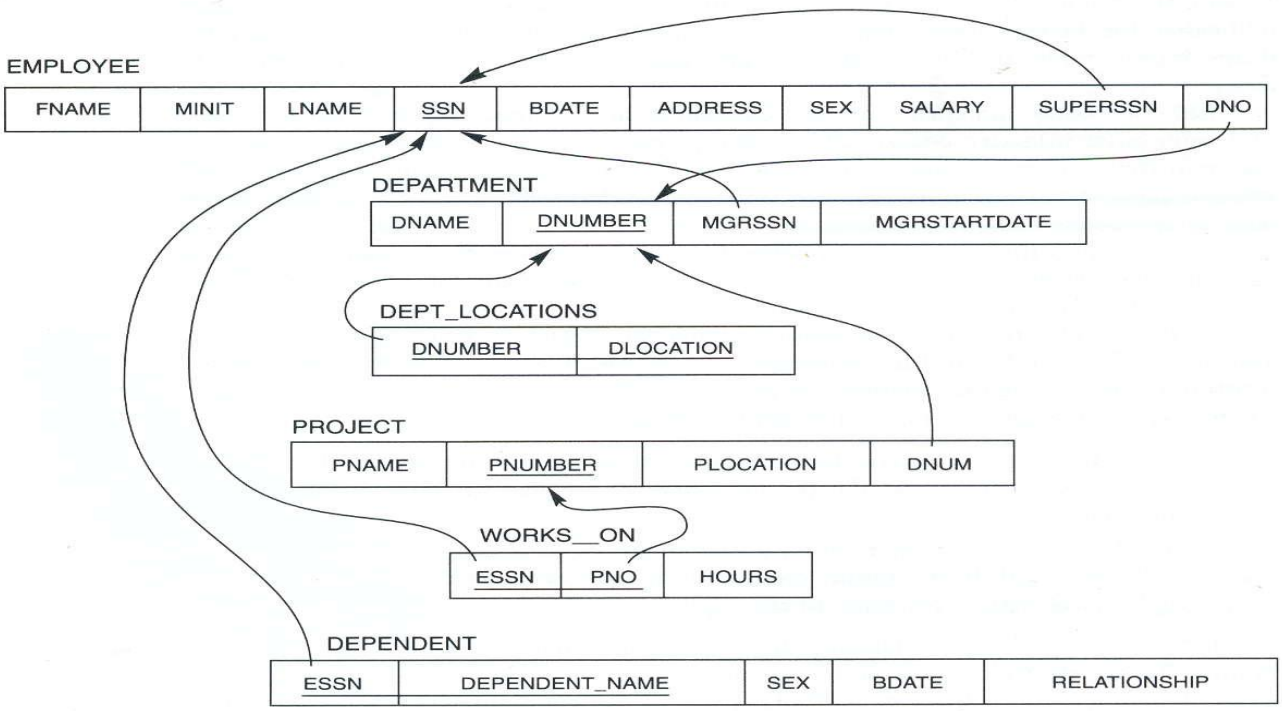
The conditions for a foreign key, specify a referential integrity constraint between the two relation schemas R1 and R2. A set of attributes FK in relation schema R1 is a **foreign key** of R1 that **references** relation R2 if it satisfies the following rules:

1. The attributes in FK have the same domain(s) as the primary key attributes PK of R2; the attributes FK are said to **reference** or **refer to** the relation R2.
2. A value of FK in a tuple t1 of the current state r1(R1) either occurs as a value of PK for some tuple t2 in the current state r2(R2) or is NULL.

i.e. t1[FK] = t2[PK], and we say that the tuple t1 **references** or **refers to** the tuple t2.

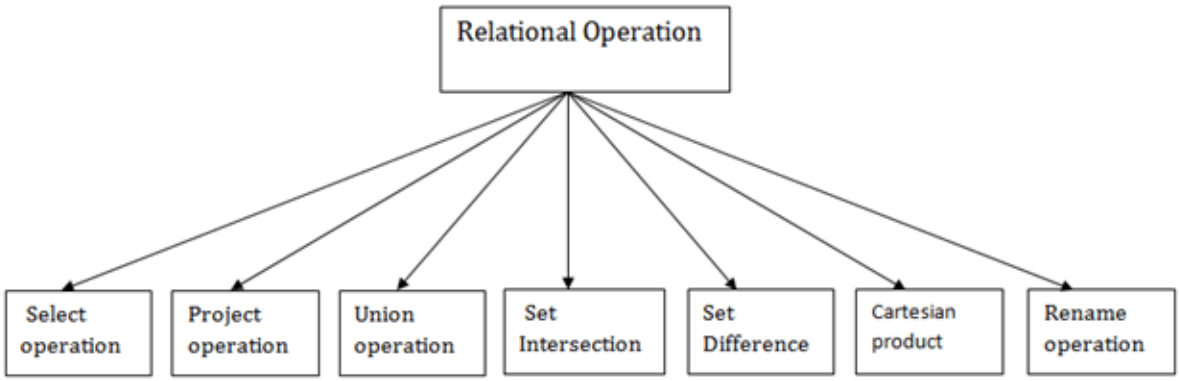
In this definition, R1 is called the **referencing relation** and R2 is the **referenced relation**.

If these two conditions hold, a **referential integrity constraint** from R1 to R2 is said to hold.



Referential integrity constraints displayed on the COMPANY relational database schema

Types of Relational operation



1. **Select Operation:**

- The select operation selects tuples that satisfy a given predicate.
- It is denoted by sigma (σ).
- Notation: σ p(r)

Where:

σ is used for selection prediction

r is used for relation

p is used as a propositional logic formula which may use connectors like: AND OR and NOT. These relational can use as relational operators like =, ≠, ≥, <, >, ≤.

For example: LOAN Relation

BRANCH_NAME	LOAN_NO	AMOUNT
Downtown	L-17	1000
Redwood	L-23	2000
Perryride	L-15	1500
Mianus	L-13	500
Roundhill	L-11	900
Perryride	L-16	1300

Input:

1. **σ BRANCH_NAME="perryride" (LOAN)**

Output:

BRANCH_NAME	LOAN_NO	AMOUNT
Perryride	L-15	1500
Perryride	L-16	1300

2. Project Operation:

- This operation shows the list of those attributes that we wish to appear in the result. Rest of the attributes are eliminated from the table.
- It is denoted by ∏.
- Notation: ∏ A1, A2, An (r)

Where

A1, **A2**, **A3** is used as an attribute name of relation **r**.

Example: CUSTOMER RELATION

NAME	STREET	CITY
Jones	Main	Harrison
Smith	North	Rye
Hays	Main	Harrison
Curry	North	Rye
Johnson	Alma	Brooklyn
Brooks	Senator	Brooklyn

Input:

1. **∏ NAME, CITY (CUSTOMER)**

Output:

NAME	CITY
Jones	Harrison
Smith	Rye
Hays	Harrison
Curry	Rye
Johnson	Brooklyn
Brooks	Brooklyn

3. Union Operation:

- Suppose there are two tuples R and S. The union operation contains all the tuples that are either in R or S or both in R & S.
- It eliminates the duplicate tuples. It is denoted by ∪.
- Notation: R ∪ S

A union operation must hold the following condition:

- R and S must have the attribute of the same number.
- Duplicate tuples are eliminated automatically.

Example:

DEPOSITOR RELATION

CUSTOMER_NAME	ACCOUNT_NO
Johnson	A-101
Smith	A-121

Mayes	A-321
Turner	A-176
Johnson	A-273
Jones	A-472
Lindsay	A-284

BORROW RELATION

CUSTOMER_NAME	LOAN_NO
Jones	L-17
Smith	L-23
Hayes	L-15
Jackson	L-14
Curry	L-93
Smith	L-11
Williams	L-17

Input:

1. **∏ CUSTOMER_NAME (BORROW) ∪ ∏ CUSTOMER_NAME (DEPOSITOR)**

Output:

CUSTOMER_NAME
Johnson
Smith
Hayes
Turner
Jones
Lindsay
Jackson
Curry
Williams
Mayes

4. Set Intersection:

- Suppose there are two tuples R and S. The set intersection operation contains all tuples that are in both R & S.
- It is denoted by intersection ∩.

1. Notation: R ∩ S

Example: Using the above DEPOSITOR table and BORROW table

Input:

1. **∏ CUSTOMER_NAME (BORROW) ∩ ∏ CUSTOMER_NAME (DEPOSITOR)**

Output:

CUSTOMER_NAME
Smith
Jones

5. Set Difference:

- Suppose there are two tuples R and S. The set intersection operation contains all tuples that are in R but not in S.
- It is denoted by intersection minus (-).
- Notation: R - S

Example: Using the above DEPOSITOR table and BORROW table

Input:

1. **∏ CUSTOMER_NAME (BORROW) - ∏ CUSTOMER_NAME (DEPOSITOR)**

Output:

CUSTOMER_NAME
Jackson
Hayes
Willians
Curry

6. Cartesian product

- The Cartesian product is used to combine each row in one table with each row in the other table. It is also known as a cross product.
- It is denoted by X.
- Notation: E X D

Example:

EMPLOYEE

EMP_ID	EMP_NAME	EMP_DEPT
1	Smith	A
2	Harry	C
3	John	B

DEPARTMENT

DEPT_NO	DEPT_NAME
A	Marketing
B	Sales
C	Legal

Input:

1. **EMPLOYEE X DEPARTMENT**

Output:

EMP_ID	EMP_NAME	EMP_DEPT	DEPT_NO	DEPT_NAME
1	Smith	A	A	Marketing
1	Smith	A	B	Sales
1	Smith	A	C	Legal
2	Harry	C	A	Marketing
2	Harry	C	B	Sales
2	Harry	C	C	Legal

3	John	B	A	Marketing
3	John	B	B	Sales
3	John	B	C	Legal

7. Rename Operation:

The rename operation is used to rename the output relation. It is denoted by **rho** (ρ).

Example: We can use the rename operator to rename STUDENT relation to STUDENT1.

ρ(STUDENT1, STUDENT)

Advantages and Disadvantages

Advantages

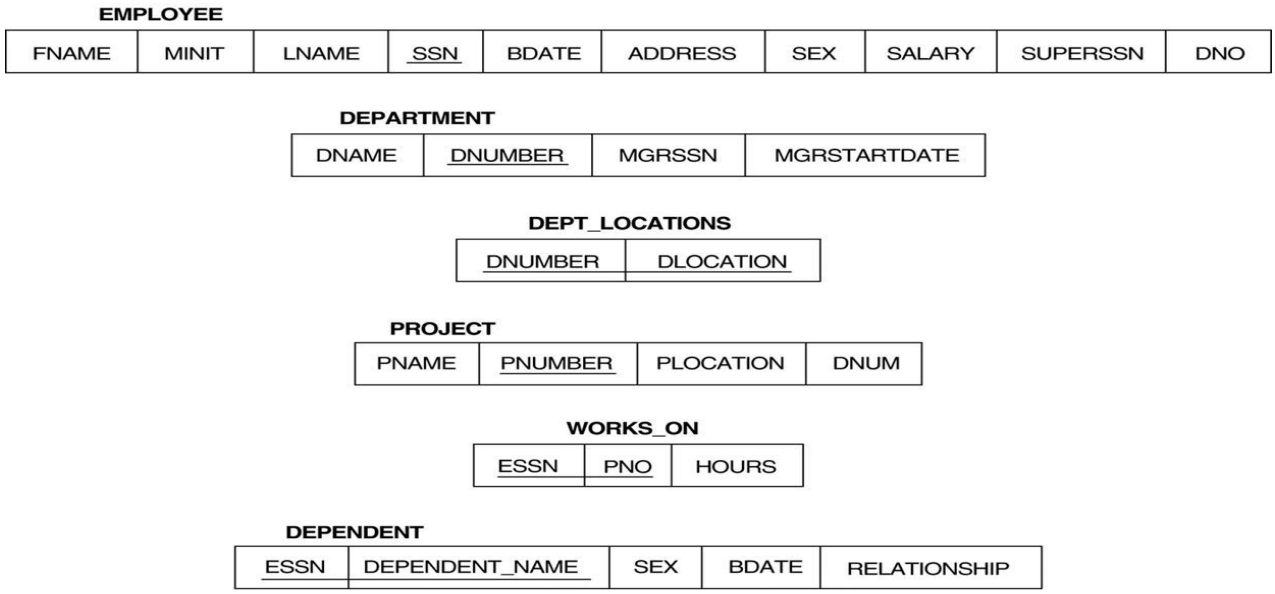
- **Simplicity**: A Relational data model in DBMS is simpler than the hierarchical and network model.
- **Structural Independence**: The relational database is only concerned with data and not with a structure. This can improve the performance of the model.
- **Easy to use**: The Relational model in DBMS is easy as tables consisting of rows and columns are quite natural and simple to understand
- **Query capability**: It makes possible for a high-level query language like SQL to avoid complex database navigation.
- **Data independence**: The Structure of Relational database can be changed without having to change any application.
- **Scalable**: Regarding a number of records, or rows, and the number of fields, a database should be enlarged to enhance its usability.

Disadvantages

- Few relational databases have limits on field lengths which can't be exceeded.
- Relational databases can sometimes become complex as the amount of data grows, and the relations between pieces of data become more complicated.
- Complex relational database systems may lead to isolated databases where the information cannot be shared from one system to another.

Relational Databases and Relational Database Schemas

- A relational database usually contains many relations, with tuples in relations that are related in various ways.
- A **relational database schema** S is a set of relation schemas S = {R1, R2, ..., Rm} and a set of **integrity constraints** IC. A **relational database state** DB of S is a set of relation states DB = {r1, r2, ..., rm} such that each ri is a state of Ri and such that the ri relation states satisfy the integrity constraints specified in IC.
- Figure 3.3 shows a relational database schema that we call COMPANY = {EMPLOYEE, DEPARTMENT, DEPT_LOCATIONS, PROJECT, WORKS_ON, DEPENDENT}.



Schema diagram for the COMPANY relational database **schema.**

Design Anomalies

An anomaly is an irregularity, or something which deviates from the expected or normal state. When designing databases, we identify three types of anomalies: Insert, Update and Delete.

The operations of the relational model can be categorized into retrievals and updates.

There are three basic operations database **modification** or **update** operations.

- **Insert,**
- **Delete, and**
- **Update (or Modify).**

Insert is used to insert one or more new tuples in a relation,

Delete is used to delete tuples,

Update (or **Modify**) is used to change the values of some attributes in existing tuples.

Whenever these operations are applied, the integrity constraints specified on the relational database schema should not be violated.

The Insert Operation

The **Insert** operation provides a list of attribute values for a new tuple t that is to be inserted into a relation R. Insert can violate any of the four types of constraints,

- Domain constraints can be violated if an attribute value is given that does not appear in the corresponding domain or is not of the appropriate data type.
- Key constraints can be violated if a key value in the new tuple t already exists in another tuple in the relation r(R).
- Entity integrity can be violated when the primary key of the new tuple t is NULL.
- Referential integrity can be violated if the value of any foreign key in t refers to a tuple that does not exist in the referenced relation.

Operation:

Insert <‘Cecilia’, ‘F’, ‘Kolonsky’, NULL, ‘1960-04-05’, ‘6357 Windy Lane, Katy, TX’, F, 28000, NULL, 4> into EMPLOYEE.

Result: This insertion violates the entity integrity constraint (NULL for the primary key Ssn), so it is rejected.

Operation:

Insert <'Alicia', 'J', 'Zelaya', '999887777', '1960-04-05', '6357 Windy Lane, Katy,TX', F, 28000, 987654321', 4> into EMPLOYEE.

Result: This insertion violates the key constraint because another tuple with the same Ssn value already exists in the EMPLOYEE relation, and so it is rejected.

Operation:

Insert <'Cecilia', 'F', 'Kolonsky', '677678989', '1960-04-05', '6357 Windswept,Katy, TX', F, 28000, '987654321', 7> into EMPLOYEE.

Result: This insertion violates the referential integrity constraint specified on Dno in EMPLOYEE because no corresponding referenced tuple exists in DEPARTMENT with Dnumber = 7.

Operation:

Insert <'Cecilia', 'F', 'Kolonsky', '677678989', '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, NULL, 4> into EMPLOYEE.

Result: This insertion satisfies all constraints, so it is acceptable.

The Delete Operation

The **Delete** operation **can** violate only referential integrity. This occurs if the tuple being deleted is referenced by foreign keys from other tuples in the database.

Operation:

Delete the WORKS_ON tuple with Essn = '999887777' and Pno = 10.

Result: This deletion is acceptable and deletes exactly one tuple.

Operation:

Delete the EMPLOYEE tuple with Ssn = '999887777'

Result: This deletion is not acceptable, because there are tuples in WORKS_ON that refer to this tuple. Hence, if the tuple in EMPLOYEE is deleted, referential integrity violations will result.

Operation:

Delete the EMPLOYEE tuple with Ssn = '333445555'.

Result: This deletion will result in even worse referential integrity violations, because the tuple involved is referenced by tuples from the EMPLOYEE, DEPARTMENT,WORKS_ON, and DEPENDENT relations.

Several options are available if a deletion operation causes a violation.

- The first option, called **restrict,** is to **reject** the deletion.
- The second option, called **cascade,** is to attempt to **cascade** (or propagate) the deletion by deleting tuples that reference the tuple that is being deleted.

For example, in operation 2, the DBMS could automatically delete the offending tuples from WORKS_ON with Essn = '999887777'.

- A third option, called **set null** or **set default,** is to modify the referencing attribute values that cause the violation; each such value is either set to NULL or changed to reference another default valid tuple.

The Update Operation

The **Update** (or **Modify**) operation is used to change the values of one or more attributes in a tuple (or tuples) of some relation R. It is necessary to specify a condition on the attributes of the relation to select the tuple (or tuples) to be modified.

Here are some examples.

Operation:

Update the salary of the EMPLOYEE tuple with Ssn = '999887777' to 28000.

Result: Acceptable.

Operation:

Update the Dno of the EMPLOYEE tuple with Ssn = '999887777' to 1.

Result: Acceptable.

Operation:

Update the Dno of the EMPLOYEE tuple with Ssn = '999887777' to 7.

Result: Unacceptable, because it violates referential integrity.

Operation:

Update the Ssn of the EMPLOYEE tuple with Ssn = '999887777' to '987654321'.

Result: Unacceptable, because it violates primary key constraint by repeating a value that already exists as a primary key in another tuple; it violates referential integrity constraints because there are other relations that refer to the existing value of Ssn.

Updating an attribute that is neither part of a primary key nor of a foreign key usually causes no problems.

The DBMS need only check to confirm that the new value is of the correct data type and domain. Modifying a primary key value is similar to deleting one tuple and inserting another in its place because we use the primary key to identify tuples.

If a foreign key attribute is modified, the DBMS must make sure that the new value refers to an existing tuple in the referenced relation.

The Transaction Concept

A transaction is an executing program that includes some database operations, such as reading from the database, or applying insertions, deletions, or updates to the database.

At the end of the transaction, it must leave the database in a valid or consistent state that satisfies all the constraints specified on the database schema.

Features of good Database Design

- We should be able to **store all kinds of data** that exist in this real world. Since we need to work with all kinds of data and requirements, the database should be strong enough to store all kinds of data that are present around us.

- We should be **able to relate the entities/tables in the database by means of relation**. i.e.; any two tables should be related. Let us say, an employee works for a department. This implies that an Employee is related to a particular department. We should be able to define such a relationship between any two entities in the database. There should not be any table lying without any mapping.

- **Data and applications should be isolated**. Because the database is a system that gives the platform to store the data, and the data is the one that allows the database to work. Hence there should be a clear differentiation between them.

- There should not be any duplication of data in the database. Data should be stored in such a way that it should not be repeated in multiple tables. If repeated, it would be an unnecessary waste of DB space, and maintaining such data becomes chaos.

- DBMS has a strong query language. Once the database is designed, this helps the user to retrieve and manipulate the data. If a particular user wants to see any specific data, he can apply as many filtering conditions that he wants and pull the data that he needs.

- Multiple users should be able to access the same database, without affecting the other user. i.e.; if teachers want to update a student's marks in the Results table at the same time, then they should be allowed to update the marks for their subjects, without modifying other subject marks. A good database should support this feature.

- It supports multiple views to the user, depending on his role. In a school database, Students will able to see only their reports and their access would be read-only. At the same time, teachers will have access to all the students with modification rights. But the database is the same. Hence a single database provides different views to different users.

- The database should also provide security, i.e.; when there are multiple users are accessing the database, each user will have their own levels of rights to see the database. Some of them will be allowed to see the whole database, and some will have only partial rights. For example, an instructor who is teaching Physics will have access to see and update marks of his subject. He will not have access to other subjects. But the HOD will have full access to all the subjects.

- The database should also support the ACID property. i.e.; while performing any transactions like insert, update and delete, the database makes sure that the real purpose of the data is not lost. For example, if a student's address is updated, then it should make sure that there is no duplicate data is created nor there is any data mismatch for that student.